

PARTE 1: Estudiar el funcionamiento de la clase “GeneradorDatosGenerico.java” y responder las preguntas presentadas en pantalla.

1) ¿Cómo se genera un vector monótonamente ascendente

Se utiliza el método "generarDatosAscendentes" de la clase GeneradorDatosGenericos".

La forma de invocar el método es, primero crear una instancia de la clase:

```
"GeneradorDatosGenericos gdg = new GeneradorDatosGenericos();"
```

y luego llamar el método que necesitamos: `"int[] vectorAscendente = gdg.generarDatosAscendentes();"`.

Como se ve, el método devuelve un array de datos de tipo entero, por lo que al hacer la llamada hay que crear un "int[]" para guardar los datos.

El método en sí es bastante sencillo.

- Primero se crea un array de tipo entero de tamaño 32000 (TAMANIO_MAX).
- Luego se recorre el array con un for loop, en donde la variable "i" va ascendiendo de a uno hasta llegar al final. (32000)
- Utilizando esa misma variable "i" se carga el valor (0,1,2,...,31999) en la posición "i" del array: `"copiaAscendente[i]"`.
- Y para finalizar, se devuelve el array "copiaAscendente".

2) ¿Cómo se genera un vector monótonamente descendente?

Se utiliza el método "generarDatosDescendentes" de la clase "GeneradorDatosGenericos".

La forma de invocar el método es, primero crear una instancia de la clase:

```
"GeneradorDatosGenericos gdg = new GeneradorDatosGenericos();"
```

y luego llamar el método que necesitamos: `int[] vectorDescendente = gdg.generarDatosDescendentes();`

Como se ve, el método devuelve un array de datos de tipo entero, por lo que al hacer la llamada hay que crear un "int[]" para guardar los datos.

Funcionamiento del método:

- Primero se crea un array de tipo entero de tamaño 32000 (TAMANIO_MAX) amdo "copiaDescendente".
- Luego se recorre el array con un for loop, donde la variable i va ascendiendo de a uno hasta llegar al final. (32000)
- Utilizando la misma variable "i" se carga el valor (TAMANIO_MAX - i) en la posición "i" del array.

Esto carga desde 32000 hasta 1, a diferencia del método ascendente que carga de 0 hasta 31999.

- Y para finalizar, se devuelve el array "copiaDescendente".

3) ¿Cómo se genera un vector con valores aleatorios? ¿pueden existir claves repetidas? ¿cuál es el orden del tiempo de ejecución de este método?

Se utiliza el método "generarDatosAleatorios" de la clase "GeneradorDatosGenericos".

La forma de invocar el método es, primero crear una instancia de la clase:
"GeneradorDatosGenericos gdg = new GeneradorDatosGenericos();"

y luego llamar el método que necesitamos: "int[] vectorAleatorio =
gdg.generarDatosAleatorios();"

Como se ve, el método devuelve un array de datos de tipo entero, por lo que al hacer la llamada hay que crear un "int[]" para guardar los datos.

Funcionamiento del método:

Primero crea una instancia de Random de java, el cual utiliza esta formula:

"this(seedUniquifier() ^ System.nanoTime());" garantizando un numero 'aleatorio'.

Luego se crean dos vectores, el primero: "int[] datosGenerados = new
int[TAMANIO_MAX];" que se usa para almacenar los valores.

Y el segundo vector: boolean[] datosUtilizados = new boolean[TAMANIO_MAX];. Que se utiliza para saber que posiciones del vector ya estan utilizadas.

EL vector se modifica utilizando un for loop, el cual va desde 0 hasta el 'length' del array de datosGenerados (32000).

Esto es lo que sucede en el for loop:

```
for (int i = 0; i < datosGenerados.length; i++) {  
    int j = rnd.nextInt(TAMANIO_MAX); // elijo una posición al azar  
    while (datosUtilizados[j]) {      // si ya está ocupada ->  
        j = (j + 1) % TAMANIO_MAX;    // -> avanzo a la siguiente (circular)  
    }  
    datosGenerados[j] = i;           // pongo el valor i ahí  
    datosUtilizados[j] = true;       // marco la posición como usada  
}
```

Para cada valor i , tira una posición al azar j .

Si esa posición ya estaba ocupada, va corriéndose linealmente hasta encontrar un hueco libre.

El $\% \text{TAMANIO_MAX}$ hace que sea circular: si llega al final, vuelve al principio.

El arreglo `datosUtilizados` es el que lleva la cuenta de qué posiciones ya se llenaron.

Y por ultimo se devuelve el vector de `'datosGenerados'`.

Claves repetidas:

El método utiliza la técnica de "sondeo lineal", la misma que se usa en la resolución de colisiones en las tablas hash. Esto garantiza que no existan repetidos.

O sea, se elige una posición al azar y si no está ocupada guardo el dato que quiero colocar (el dato no se repite por el for loop).

Si esa posición está ocupada paso a la siguiente, utilizando el while loop, y así hasta encontrar una vacía.

Tiempo de ejecución:

```
for (int i = 0; i < datosGenerados.length; i++) { // se repite n veces
    int j = rnd.nextInt(TAMANIO_MAX);
    while (datosUtilizados[j]) { ... } // cada vez puede costar hasta O(n)
    ...
}
```

Con este ejemplo podemos ver que una sola inserción, en el peor caso, cuesta $O(n)$ (cuando el sondeo lineal tiene que recorrer casi todo el arreglo para hallar el único hueco).

Como esa inserción se ejecuta n veces dentro del for, el método completo es $O(n^2)$ en el peor caso.

4) ¿Cuántos elementos contiene el vector de datos generado? ¿cómo se puede modificar esta clase

para que la cantidad de elementos del vector sea parametrizable?

Contienen 32000 elementos.

Esto se puede modificar parametrizando la variable "TAMANIO_MAX" para poder pasarle el valor que queramos al instanciar la clase.

Primero debería dejar de ser estática y final, y posiblemente cambiarle el nombre a algo como "tamanio" para que se sepa que no es final.

(obviamente también cambiar el nombre en los métodos).

Y luego generar un constructor que reciba el input del usuario para crear la clase con ese valor:

```
public GeneradorDatosGenericos(int tamanio) {  
    this.tamanio = tamanio;  
}
```

Otra forma sería eliminar la constante "TAMANIO_MAX" y pasarle a cada método el valor que queramos para el vector.

5) ¿Cómo podemos verificar que un conjunto está ordenado? ¿cuál sería el orden del tiempo de ejecución de un algoritmo que lo haga?

La forma más sencilla de hacerlo es creando un for loop y utilizando la transitividad de los

valores, es decir, si $\text{vector}[i] > \text{vector}[i+1]$ ya sabemos que no está ordenado.

El método podría ser algo como:

```
public boolean estaOrdenado(int[] vector) {  
    for (int i = 0; i < vector.length - 1; i++) {  
        if (vector[i] > vector[i + 1]) {  
            return false;  
        }  
    }  
    return true; // llegué al final -> está ordenado  
}
```

Para orden descendente es lo mismo solo se da vuelta la comparación: if ($v[i] < v[i + 1]$).

Tiempo de ejecución:

$O(n)$ en el peor caso. Si se comparase todos los valores contra todos los valores, entonces sí, sería $O(n^2)$,

pero como no lo hacemos, el peor caso es recorrer toda la lista (si está ordenada) y eso nos da $O(n)$.